Challenge Description

This challenge aims to create a task management web application (ToDo List) that allows users to authenticate, create, edit, delete, and view tasks. The application should be developed using the MERN stack (MongoDB, Express, React, Node.js) and meet the specified requirements.

Requirements

Backend (API with Node.js and Express)

1. **Initial Setup**:
   - Use TypeScript for backend development.
   - Set up an Express server.
   - Use Sequelize for PostgreSQL database management.

2. **Authentication**:
   - Implement user authentication with Auth0.
   - Protect routes with authentication middleware.

3. **Routes**:
   - `POST /api/todos`: Create a new task.
   - `GET /api/todos`: Retrieve all tasks for the authenticated user.
   - `GET /api/todos/:id`: Retrieve a specific task by ID.
   - `PUT /api/todos/:id`: Update a task by ID.
   - `DELETE /api/todos/:id`: Delete a task by ID.

4. **Validation and Error Handling**:
   - Use Zod to validate input data.
   - Implement global error handling.

5. **API Documentation** (optional)
   - Use Swagger/others to document the API.

6. **Critical Test**:
   - Implement a unit test for task creation, verifying tasks are created correctly and the response includes expected data.

Frontend (React Application with Next.js)

1. **Initial Setup**:
   - Set up a Next.js project with TypeScript.
   - Use Styled Components (library) for styling or other ui libraries.
   - Configure ESLint and Prettier for code consistency and cleanliness.

2. **Authentication**:
   - Implement authentication with Auth0 in the front end.
   - Use Redux Toolkit to manage the global authentication state.

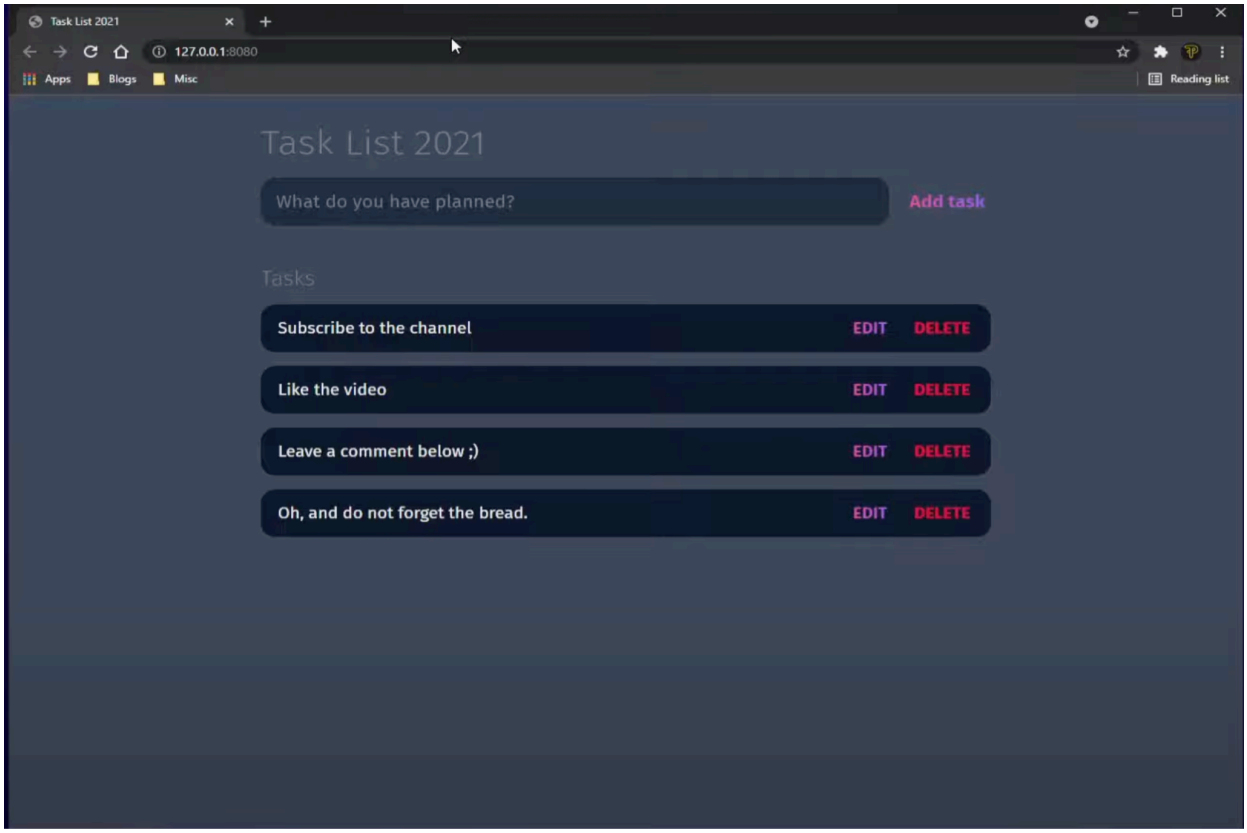
3. **Task Management**:
   - Use Axios/fetch to consume the API.
   - Implement Redux Toolkit to manage data fetching state.
   - Allow users to create, edit, delete, and view tasks.

4. **UI/UX**:
   - Use Material-UI (MUI) for user interface components or other.
   - Implement a form for task management using React-hook-form.

5. **State Persistence**:
   - Use Redux Toolkit to persist the global state.
   - Store data locally with LocalStorage (plus)

6. **Critical Test**:
   - Implement a unit test for the task creation form, verifying it renders correctly and required fields are present.

Proposed Design



Additional Requirements

1. **Testing**:
   - Write a critical unit test for the backend.
   - Write a critical unit test for the front end.

2. **Deployment** (optional)
   - Provide clear instructions for deploying the application to a production environment.
   - Include a link where the application is running online.

3. **Documentation** (readme)
   - Document the setup and for running the project.
   - Include comments in the code to explain design decisions.

Evaluation Criteria

1. **Functionality**: The application meets all specified functional requirements.
2. **Code Quality**: The code is clean, well-organized, and follows best practices.

3. **Authentication and Security**: Authentication is well-implemented and routes are protected.
4. **Tests**: The application includes the specified critical unit tests.
5. **Documentation**: The documentation is clear and complete.
6. **Deployment (optional)**: The application can be deployed following the instructions and is available online. Use separate repositories for Backend and Frontend.
7. **Important!!** Write the entire application code in English.

Submission

The candidate must submit:

1. The source code of the project in a GitHub repository.
2. A README file with instructions for setting up and deploying the application.
3. A link to the live application (optional)

This challenge is designed to be completed within 15 days and will evaluate the candidate's technical skills and ability to follow good development practices. Good luck!

Proposed Libraries

- **TypeScript: https://www.typescriptlang.org/**
- Express: https://expressjs.com/
- Sequelize: https://sequelize.org/
- PostgreSQL: https://www.postgresql.org/
- Auth0: https://auth0.com/
- Zod: https://zod.dev/
- Swagger: https://swagger.io/
- Next.js: https://nextjs.org/
- Styled Components: https://styled-components.com/
- ESLint: https://eslint.org/
- Prettier: https://prettier.io/
- Redux Toolkit: https://redux-toolkit.js.org/
- Axios: https://axios-http.com/
- React Query: https://tanstack.com/query/latest
- Material-UI (MUI): https://mui.com/
- React Hook Form: https://react-hook-form.com/
- Redux Persist: https://github.com/rt2zz/redux-persist
- Localforage: https://localforage.github.io/localForage/